PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
46504-04 · Construção de Software · Turma 330 · 2026/2
Prof. Michael da Costa Mora
PROJETO INTEGRADOR · PACOTE 1

PulsePass

Plataforma Distribuída de Bilhetagem, Reserva e Acesso para Megaeventos

Processo de Desenvolvimento e Requisitos

Processo com IA (SDD)Visão do ProdutoStory MappingRNFsMockups e Navegação

Integrantes: Gabriel Kowaleski e Mayra Bordin

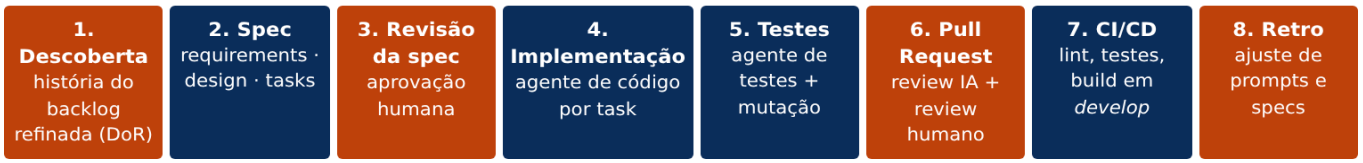
Entrega: 14 de outubro de 2026

Sumário. 1. Processo de Desenvolvimento Suportado por IA · 2. Documento de Visão do Produto · 3. Backlog: Épicos, Histórias e Story Map · 4. Requisitos Não-Funcionais · 5. Mockups e Mapa de Navegação do Front-end

1. Processo de Desenvolvimento Suportado por IA

O grupo adota um processo iterativo e incremental baseado em **Spec-Driven Development (SDD)**: toda funcionalidade nasce de uma especificação escrita e revisada por humanos, que vira o contrato para os agentes de IA gerarem código, testes e documentação. A IA acelera a execução; a decisão (o que construir, aceitar ou rejeitar) permanece com o time. O ciclo é organizado em **sprints de duas semanas** alinhadas aos quatro pacotes de entrega da disciplina.

1.1 Ciclo de desenvolvimento



Laranja = etapa com decisão humana obrigatória (*human-in-the-loop*). Azul = etapa executada majoritariamente por agentes, sempre sobre a spec aprovada.

1. **Descoberta.** O Product Owner do grupo seleciona histórias do Story Map (seção 3) que atendem à *Definition of Ready*: critérios de aceite escritos, dependências identificadas e mockup associado.
2. **Especificação.** Para cada história/feature é criada uma pasta em `/specs` com três arquivos: `requirements.md` (histórias e critérios de aceite em formato EARS/Gherkin), `design.md` (contratos de API, eventos Kafka, modelo de dados, decisões) e `tasks.md` (passos pequenos e verificáveis). O rascunho é gerado pelo agente de especificação a partir da história e dos RNFs.
3. **Revisão da spec.** Um integrante diferente do autor revisa e aprova a spec via PR. Spec aprovada = contrato; mudanças posteriores voltam por esta etapa.
4. **Implementação.** O agente de código executa as tasks uma a uma em branch `feature/*`, com contexto restrito à spec e ao módulo (microsserviço) correspondente.
5. **Testes.** O agente de testes gera testes unitários a partir dos critérios de aceite (foco nas máquinas de estado) e testes de integração com Testcontainers (PostgreSQL, Kafka, Redis). Teste de mutação (PIT) é usado para validar a qualidade dos testes gerados.
6. **Pull Request.** O agente de revisão comenta o PR (padrões, segurança, aderência à spec); a aprovação final e o merge em `develop` são feitos por um humano.
7. **CI/CD.** GitHub Actions executa lint, testes e build a cada PR/merge em `develop`; o ambiente sobe completo por `docker-compose`.

8. **Retrospectiva.** Ao fim da sprint, o grupo revisa métricas (lead time, retrabalho, bugs que escaparam) e atualiza o arquivo de regras dos agentes e os templates de spec.

1.2 Ferramentas

Finalidade	Ferramenta	Uso no projeto
Assistente/agente de código	Claude Code (CLI) e GitHub Copilot (IDE)	Geração de specs, implementação por task, testes, refatoração e revisão de PR
Gestão do backlog	GitHub Projects (quadro Kanban + Story Map)	Épicos como *milestones*, histórias como *issues* com rótulos de release
Versionamento e GCS	Git + GitHub (Git Flow simplificado)	`main`, `develop`, `feature/*`, `fix/*`; PR obrigatório
CI/CD	GitHub Actions + Docker Compose	Lint, testes unitários/integração, build de imagens
Design / UX	Figma	Mockups (seção 5) e protótipo navegável
Documentação arquitetural	Markdown + Mermaid / Structurizr (C4)	Diagramas versionados junto ao código (Pacote 2)
Qualidade	JUnit 5, Testcontainers, PIT, SonarLint	Cobertura > 70% do domínio, testes de integração e mutação

1.3 Agentes e responsabilidades

Agente	Entrada → Saída	Guarda-corpos
Spec Agent	História + RNFs → `requirements.md`, `design.md`, `tasks.md`	Não inventa regra de negócio: marca dúvidas como `[ABERTO]` para o time decidir
Code Agent	Task aprovada → código em branch `feature/*`	Escopo limitado ao serviço da task; sem acesso a segredos; commits pequenos
Test Agent	Critérios de aceite → testes unitários e de integração	Testes devem falhar antes da implementação; mutação ≥ 60% nos módulos críticos
Review Agent	Diff do PR → comentários de revisão	Apenas sugere; nunca aprova nem faz merge
Docs Agent	Código + specs → README, ADRs e diagramas C4	Documentação revisada no mesmo PR da mudança

1.4 Estrutura das specs no repositório

```
/specs
  /iam/login-jwt/            requirements.md · design.md · tasks.md
  /inventario/reserva-assento/
  /checkout/idempotencia/
  /emissao/ingresso-pdf-qr/
  /telemetria/painel-tempo-real/
/.claude/ ou /.github/       regras dos agentes (padrões de código, arquitetura, proibições)
```

/docs/adr/	decisões arquiteturais (ADR-001, ADR-002, ...)

Regras do time para uso de IA. (1) Nenhum código entra em `develop` sem revisão humana. (2) Prompts e regras dos agentes são versionados no repositório. (3) Dados pessoais reais e segredos nunca vão para prompts. (4) Todo PR indica quais partes foram geradas por IA. (5) *Definition of Done*: spec atualizada, testes passando no CI, cobertura mantida, PR revisado e documentação ajustada.

2. Documento de Visão do Produto

2.1 Problema

Na abertura de vendas de megaeventos (shows, finais esportivas, festivais), milhares de compradores acessam a plataforma no mesmo segundo. Sistemas baseados apenas em HTTP síncrono esgotam conexões de banco, ficam lentos ou fora do ar e, no pior caso, **vendem o mesmo assento duas vezes**. Para o comprador isso gera frustração e desconfiança; para o organizador, prejuízo financeiro, reembolsos e dano à marca.

2.2 Declaração de posicionamento

Para organizadores de grandes eventos e seus compradores **que** precisam vender e controlar o acesso a milhares de ingressos em poucos minutos, **o PulsePass** é uma plataforma distribuída de bilhetagem **que** garante zero sobre-venda, reserva temporária justa de 10 minutos e emissão segura de ingressos mesmo sob rajadas de tráfego. **Diferente de** sistemas monolíticos síncronos, **nosso produto** usa microsserviços orientados a eventos (Kafka), bloqueio atômico de assentos e checkout idempotente, mantendo a experiência estável no pico.

2.3 Stakeholders e usuários

Perfil	Descrição	Principais necessidades
Cliente (comprador)	Público final que compra ingressos, geralmente pelo celular e no pico de abertura de vendas.	Ver disponibilidade real, garantir o assento, pagar sem cobrança duplicada, receber o ingresso na hora.
Organizador	Produtora responsável pelo evento.	Cadastrar eventos, lotes e mapas de assentos; acompanhar vendas em tempo real.
Operador	Equipe de portaria/controle de acesso no dia do evento.	Validar QR Codes rapidamente e impedir uso duplicado de ingressos.
Administrador	Equipe da plataforma PulsePass.	Aprovar organizadores, gerir papéis, auditar acessos e monitorar a saúde do sistema.
Sistemas externos	Gateway de pagamento (simulado) e serviço de e-mail/notificação.	Integração assíncrona e tolerante a falhas.

2.4 Funcionalidades principais (por módulo)

Módulo	Capacidades
Identidade e Governança (IAM)	Login com JWT e refresh token (Keycloak), RBAC com 4 papéis, cadastro e aprovação de organizadores, auditoria de credenciais.
Inventário e Concorrência	Eventos, lotes e mapas de assentos; catálogo de alta leitura (cache); bloqueio e liberação atômica de assentos; eventos de expiração de reserva.
Transações e Checkout	Máquina de estados do pedido (Criado → Reservado → Aguardando → Pago / Expirado), `Idempotency-Key`, Transactional Outbox, integração com gateway simulado.

Emissão e Notificações	Consumo de pagamentos confirmados, ingresso em PDF com QR Code e carimbo de tempo, envio por e-mail, atualização de tela em tempo real (WebSocket).
Telemetria e DevOps	Ingestão de eventos de navegação e compra, painel de métricas em tempo real, CI/CD e orquestração com Docker Compose.

2.5 Escopo

Dentro do escopo: venda de ingressos com e sem assento marcado, reserva com expiração, pagamento via gateway simulado, emissão e validação de ingresso, painéis do organizador e operacional.

Fora do escopo: integração com gateways de pagamento reais, revenda/transferência entre usuários, aplicativo móvel nativo (o front-end será SPA responsiva), meia-entrada com validação documental e emissão fiscal.

2.6 Métricas de sucesso

- **0** casos de sobre-venda em teste de carga com concorrência simulada no mesmo assento.
- 100% das reservas não pagas liberadas em até 10 min + 30 s.
- 0 cobranças duplicadas em requisições repetidas com a mesma `Idempotency-Key`.
- Ingresso disponível ao comprador em até 30 s após a confirmação do pagamento (p95).
- Validação de ingresso na portaria em menos de 1 s (p95).

2.7 Premissas e restrições

- Arquitetura em microsserviços; Keycloak (ou Cognito) para identidade; Kafka para streaming; PostgreSQL e Redis para persistência e bloqueios.
- Ambiente completo executável localmente via `docker-compose`; CI no GitHub Actions.
- Grupo de 3 a 5 pessoas, prazo de um semestre, entregas em quatro pacotes.
- Pagamento é simulado: o gateway responde com aprovação, recusa ou atraso configuráveis para testar resiliência.

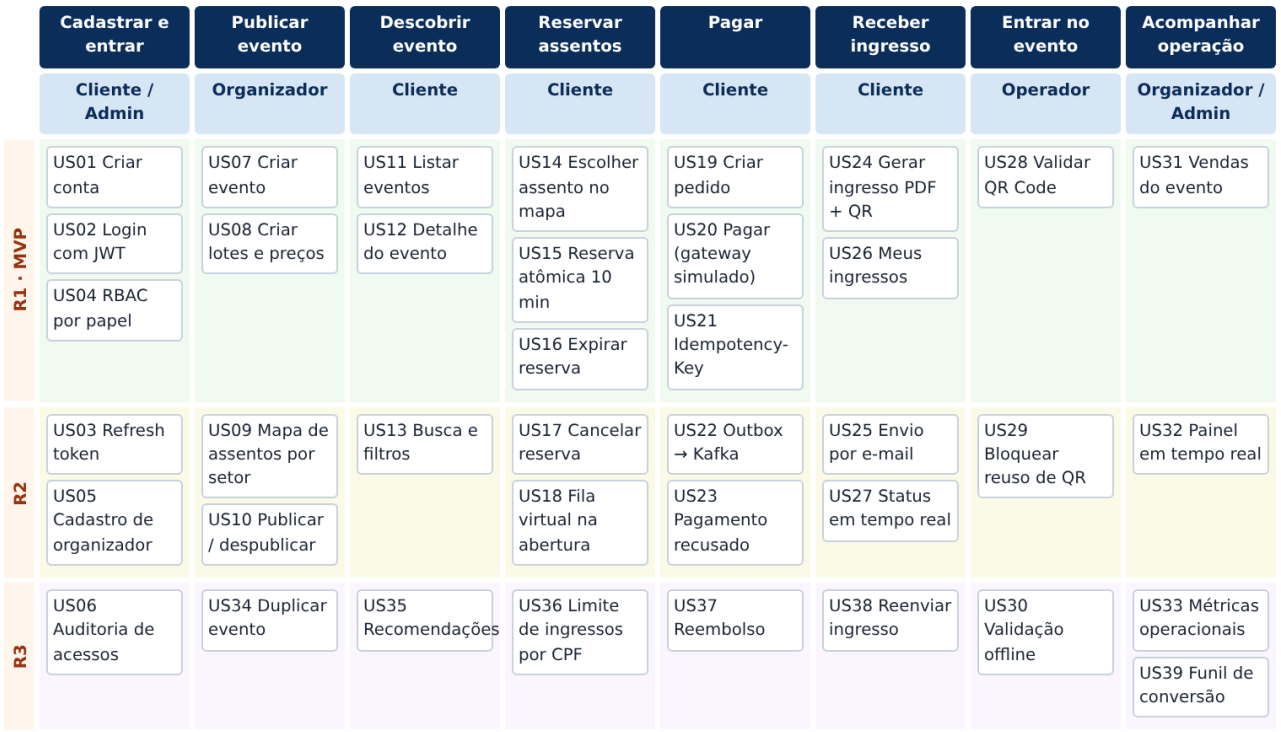
3. Backlog: Épicos, Histórias e Story Map

3.1 Épicos

ID	Épico	Objetivo	Módulo
EP1	Identidade e Acesso	Usuários se autenticam com segurança e acessam apenas o que seu papel permite.	IAM
EP2	Gestão de Eventos	Organizadores publicam eventos, lotes e mapas de assentos.	Inventário
EP3	Descoberta e Reserva	Compradores encontram eventos e reservam assentos sem conflito.	Inventário
EP4	Checkout e Pagamento	Pedidos são pagos uma única vez, com estados consistentes.	Checkout
EP5	Emissão e Acesso	Ingressos são emitidos, entregues e validados na entrada.	Emissão
EP6	Operação e Telemetria	Organizadores e admins acompanham vendas e saúde do sistema.	Telemetria

3.2 Story Map

Leitura: a linha superior é o *backbone* (jornada do usuário, da esquerda para a direita); abaixo, as histórias de cada etapa, divididas horizontalmente por release. R1 = MVP (fluxo fim a fim mínimo), R2 = robustez e experiência, R3 = operação e analytics.



3.3 Histórias de usuário

ID	Épico	História	Rel.
US01	EP1	Como **cliente**, quero criar uma conta com e-mail e senha, para poder comprar ingressos.	R1
US02	EP1	Como **usuário**, quero entrar na plataforma e receber um token de acesso, para usar as funções do meu perfil sem logar a cada tela.	R1
US03	EP1	Como **usuário**, quero que minha sessão seja renovada automaticamente, para não perder uma compra por expiração do token.	R2
US04	EP1	Como **administrador**, quero que cada rota exija o papel adequado (Cliente, Organizador, Operador, Admin), para proteger as operações sensíveis.	R1
US05	EP1	Como **organizador**, quero solicitar meu cadastro com dados da empresa, para que um admin aprove meu acesso.	R2
US06	EP1	Como **administrador**, quero consultar o histórico de logins e alterações de credenciais, para auditar acessos suspeitos.	R3
US07	EP2	Como **organizador**, quero cadastrar um evento com nome, local, data e imagem, para divulgá-lo na plataforma.	R1
US08	EP2	Como **organizador**, quero criar lotes com preço, quantidade e período de venda, para controlar a precificação.	R1
US09	EP2	Como **organizador**, quero definir setores e assentos numerados, para vender lugares marcados.	R2
US10	EP2	Como **organizador**, quero publicar ou despublicar um evento, para controlar quando as vendas abrem.	R2
US11	EP3	Como **cliente**, quero ver a lista de eventos disponíveis, para escolher o que me interessa.	R1
US12	EP3	Como **cliente**, quero ver detalhes, lotes e disponibilidade de um evento, para decidir a compra.	R1

US13	EP3	Como **cliente**, quero buscar por nome, cidade e data, para encontrar eventos rapidamente.	R2
US14	EP3	Como **cliente**, quero escolher assentos em um mapa que mostra o que está livre, reservado ou vendido, para escolher meu lugar.	R1
US15	EP3	Como **cliente**, quero que os assentos escolhidos fiquem reservados para mim por 10 minutos, para concluir o pagamento com calma.	R1
US16	EP3	Como **plataforma**, quero liberar automaticamente reservas não pagas após 10 minutos, para devolver os assentos à venda.	R1
US17	EP3	Como **cliente**, quero cancelar minha reserva, para liberar os assentos se desistir.	R2
US18	EP3	Como **cliente**, quero entrar em uma fila virtual quando a demanda estiver alta, para ter uma chance justa de comprar.	R2
US19	EP4	Como **cliente**, quero revisar meu pedido com itens e total, para confirmar antes de pagar.	R1
US20	EP4	Como **cliente**, quero pagar meu pedido, para garantir os ingressos.	R1
US21	EP4	Como **cliente**, quero que um clique repetido em "Pagar" não gere duas cobranças, para não ser cobrado em dobro.	R1
US22	EP4	Como **plataforma**, quero publicar o evento de pedido pago pelo padrão Outbox, para que nenhum pagamento confirmado deixe de gerar ingresso.	R2
US23	EP4	Como **cliente**, quero ser avisado se o pagamento for recusado e poder tentar de novo dentro do prazo da reserva.	R2
US24	EP5	Como **cliente**, quero receber meu ingresso em PDF com QR Code após o pagamento, para apresentar na entrada.	R1
US25	EP5	Como **cliente**, quero receber o ingresso por e-mail, para ter uma cópia fora da plataforma.	R2
US26	EP5	Como **cliente**, quero ver todos os meus ingressos em uma página, para acessá-los no dia do evento.	R1
US27	EP5	Como **cliente**, quero ver o status do pedido mudar em tempo real (aguardando → pago → emitido), sem recarregar a página.	R2
US28	EP5	Como **operador**, quero ler o QR Code do ingresso e ver se é válido, para liberar a entrada.	R1
US29	EP5	Como **operador**, quero que um ingresso já utilizado seja recusado, para evitar fraudes.	R2
US31	EP6	Como **organizador**, quero ver total vendido e ocupação por lote, para acompanhar meu evento.	R1
US32	EP6	Como **organizador**, quero um painel atualizado em tempo real durante a abertura das vendas, para reagir rapidamente.	R2
US33	EP6	Como **administrador**, quero ver métricas operacionais (latência, erros, lag do Kafka), para garantir a saúde da plataforma.	R3

Histórias de R3 não detalhadas acima (US30, US34–US39) permanecem no backlog como candidatas e serão refinadas conforme a capacidade do grupo.

3.4 Critérios de aceite das histórias críticas

US15 — Reserva atômica de assentos

```
Dado que o assento A10 está LIVRE

Quando dois clientes tentam reservá-lo ao mesmo tempo

Então apenas um recebe a reserva (status RESERVADO, expira em 10 min)
```

E o outro recebe HTTP 409 "assento indisponível"

E o mapa de ambos mostra A10 como reservado em até 2 s

US16 — Expiração de reserva

Dado um pedido RESERVADO há 10 minutos sem pagamento

Quando o prazo expira

Então o pedido passa para EXPIRADO

E os assentos voltam para LIVRE

E o evento "reserva.expirada" é publicado no Kafka

US21 — Pagamento idempotente

Dado um pedido AGUARDANDO pagamento

Quando o cliente envia POST /pagamentos duas vezes com a mesma Idempotency-Key

Então o gateway é chamado apenas uma vez

E a segunda resposta é idêntica à primeira

E existe somente um registro de pagamento

US28/US29 — Validação na portaria

Dado um ingresso EMITIDO e nunca utilizado

Quando o operador lê o QR Code

Então a tela mostra "Entrada liberada" com nome do evento e setor

E o ingresso passa para UTILIZADO

Quando o mesmo QR é lido novamente

Então a tela mostra "Ingresso já utilizado" com horário da primeira leitura

4. Requisitos Não-Funcionais

Os requisitos abaixo são mensuráveis e serão verificados por testes automatizados, testes de carga ou inspeção no CI. Valores de carga referem-se ao ambiente local em Docker Compose e servem como meta do projeto acadêmico.

ID	Categoria	Requisito	Critério de verificação
RNF01	Consistência	O sistema não pode vender o mesmo assento ou ultrapassar a capacidade de um lote (zero oversell).	Teste concorrente com 500 requisições simultâneas no mesmo assento → exatamente 1 sucesso.
RNF02	Consistência	Reservas não pagas expiram em 10 min e liberam os assentos.	Liberação em até 10 min + 30 s em 100% dos casos de teste.
RNF03	Confiabilidade	Operações de pagamento são idempotentes via cabeçalho `Idempotency-Key`.	Requisições repetidas não geram cobrança nem registro duplicado.
RNF04	Confiabilidade	Eventos de domínio são publicados pelo padrão Transactional Outbox (sem dual-write).	Queda do Kafka durante o pagamento não perde eventos; publicação ocorre ao retornar.

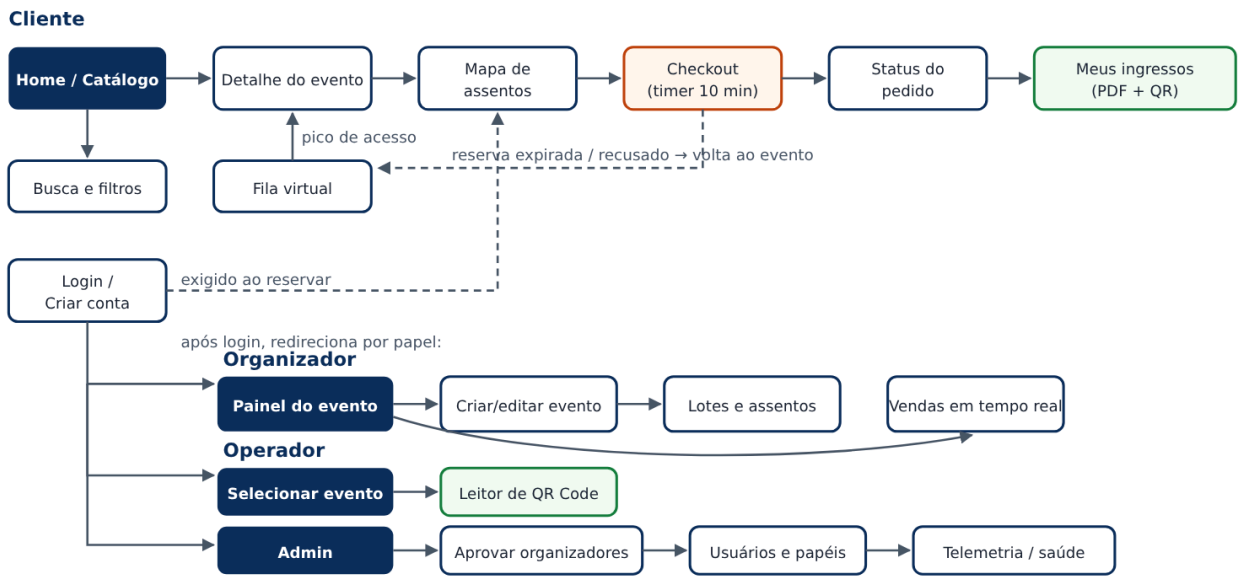
RNF05	Confiabilidade	Consumidores Kafka são idempotentes (deduplicação por ID do evento).	Reprocessar o mesmo evento não gera ingresso duplicado.
RNF06	Desempenho	Catálogo de eventos responde em p95 ≤ 200 ms sob 1.000 req/s (leitura via cache Redis).	Teste de carga (k6/Gatling) — bônus.
RNF07	Desempenho	Reserva de assento responde em p95 ≤ 500 ms sob 200 reservas/s.	Teste de carga — bônus.
RNF08	Desempenho	Ingresso emitido em até 30 s (p95) após confirmação do pagamento.	Medição pelo painel de telemetria.
RNF09	Escalabilidade	Serviços sem estado em memória, podendo ter múltiplas réplicas; tópicos Kafka particionados por evento.	Subir 2 réplicas no Compose sem erro funcional.
RNF10	Resiliência	Chamadas ao gateway usam timeout, retentativa com backoff e circuit breaker.	Gateway simulado com falha/atraso não derruba o checkout.
RNF11	Disponibilidade	Falha do serviço de Emissão ou Notificação não impede a venda; mensagens ficam no Kafka até o reprocessamento.	Teste derrubando o serviço durante compras.
RNF12	Segurança	Autenticação stateless com JWT assinado (access token 15 min, refresh token 7 dias armazenado em chave-valor) via Keycloak.	Token expirado/adulterado → HTTP 401.
RNF13	Segurança	Autorização RBAC em todas as rotas (Cliente, Organizador, Operador, Admin).	Testes de integração por papel → HTTP 403 quando indevido.
RNF14	Segurança	QR Code contém identificador assinado, impossível de forjar a partir de dados públicos.	QR alterado é rejeitado na validação.
RNF15	Privacidade	Dados pessoais tratados conforme a LGPD: mínimo necessário, senhas apenas no IdP, logs sem dados sensíveis.	Revisão de código e de logs.
RNF16	Observabilidade	Logs estruturados com *correlation-id*, métricas e rastreamento distribuído (OpenTelemetry, Prometheus, Grafana).	Uma compra é rastreável ponta a ponta em um único trace.
RNF17	Manutenibilidade	Cobertura de testes > 70% no domínio (máquinas de estado); testes de integração com Testcontainers (PostgreSQL, Kafka, Redis).	Relatório de cobertura no CI.
RNF18	Manutenibilidade	Código passa por linter e revisão; arquitetura documentada em C4 e ADRs.	Pipeline falha se o lint falhar.
RNF19	Portabilidade	Todo o ambiente sobe com um único `docker compose up` (backend, frontend, bancos, Kafka e UI de mensageria).	Execução em máquina limpa.
RNF20	Integração contínua	Pipeline no GitHub Actions executa lint, testes e build em cada PR e merge na branch `develop`.	PR bloqueado se o pipeline falhar.

RNF21	Usabilidade	SPA responsiva (360 px a 1920 px); compra concluída em no máximo 5 telas a partir do detalhe do evento.	Teste com usuários / inspeção de fluxo.
RNF22	Acessibilidade	Front-end segue WCAG 2.1 nível AA (contraste, navegação por teclado, textos alternativos); mapa de assentos com alternativa em lista.	Auditoria com Lighthouse/axe ≥ 90.
RNF23	Equidade	Na abertura de vendas, o acesso ao checkout respeita ordem de chegada (fila virtual) quando a capacidade é excedida.	Teste de carga com fila ativa.

5. Mockups e Mapa de Navegação do Front-end

Mockups em média fidelidade (wireframes) da SPA. Cores e tipografia são provisórias; o foco é estrutura, conteúdo e fluxo. Cada tela indica as histórias que atende.

5.1 Mapa de navegação



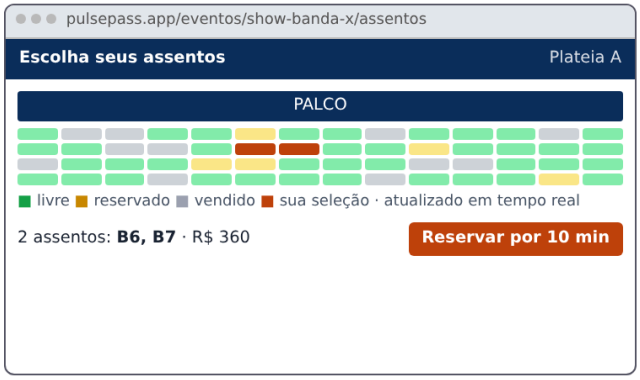
Azul escuro = ponto de entrada de cada papel após o login. Laranja = tela com tempo limitado. Verde = telas de ingresso. Linhas tracejadas = desvios do fluxo principal.

5.2 Telas do cliente



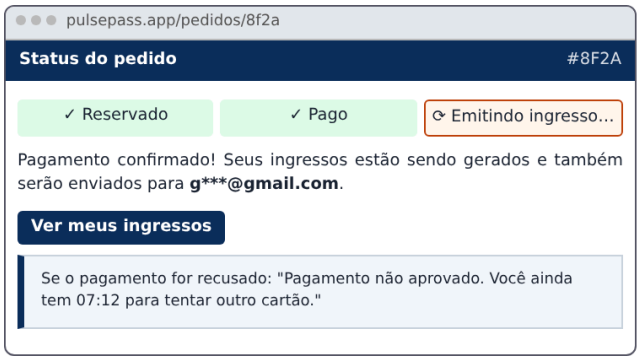


Tela 1 — Home/Catálogo (US11, US13) · **Tela 2 — Detalhe do evento** (US12): disponibilidade por lote lida do cache.





Tela 3 — Mapa de assentos (US14, US15): estados atualizados via WebSocket; conflito retorna mensagem "assento acabou de ser reservado". **Tela 4 — Checkout** (US17, US19–US21): contagem regressiva sincronizada com o servidor.





Tela 5 — Status do pedido (US23, US27): progresso em tempo real. **Tela 6 — Meus ingressos** (US24–US26): QR Code e download do PDF.

5.3 Telas do organizador, operador e administrador





Tela 7 — Painel do organizador (US07–US10, US31, US32). **Tela 8 — Leitor de QR do operador** (US28, US29), otimizada para celular.





Tela 9 — Cadastro de evento em etapas (US07–US09). **Tela 10 — Administração** (US04–US06, US33).